Can AI Agents Generate Microservices? How Far are We?

Anonymous Author(s)

Abstract—Context. **LLMs have advanced code generation, but their use for generating microservices with explicit dependencies and API contracts remains understudied.**

Goal. **We examine whether AI agents can generate functional microservices and how different forms of contextual information influence their performance.**

Method. **We assess 144 generated microservices across 3 agents, 4 projects, 2 prompting strategies, and 2 scenarios. Incremental generation operates within existing systems and is evaluated with unit tests. Clean state generation starts from requirements alone and is evaluated with integration tests. We analyze functional correctness, code quality, and efficiency.**

Results. **Minimal prompts outperformed detailed ones in incremental generation, with 50–76% unit test pass rates. Clean state generation produced higher integration test pass rates (81–98%), indicating strong API contract adherence. Generated code showed lower complexity than human baselines. Generation times varied widely across agents, averaging 6–16 minutes per service.**

Conclusions. **AI agents can produce microservices with maintainable code, yet inconsistent correctness and reliance on human oversight show that fully autonomous microservice generation is not yet achievable.**

Index Terms**—Software Architecture, Microservices, AI agents**

I. Introduction

Automating the generation of executable systems from architecture descriptions has long been a central ambition in Software Architecture (SA), offering benefits such as stronger compliance and improved traceability [1]. The SA community has explored this vision through Architecture Description Languages (ADLs) and Domain-Specific Languages (DSLs) [2], yet adoption has remained limited due to steep learning curves and insufficient tool support [3].

Recent advances in AI offer a new direction. Large Language Models (LLMs) now perform a wide range of Software Engineering (SE) tasks [4], including code completion, function generation, and program repair [5]–[8]. Research on GenAI for software architecture has begun to explore requirements-to-architecture and architecture-to-code transitions, yet only few studies attempt the full requirements-to-architecture-to-code pipeline, and most lack robust validation strategies for AI-generated artifacts [9]. Work on generating larger architectural units remains scarce, with early efforts emerging in specific contexts such as serverless functions [10].

A broader shift is now occurring from standalone LLMs to AI agents. These agents augment foundation models with tool usage, memory, and iterative refinement workflows [11], [12]. Unlike single-pass LLMs, agents can run tests, analyze compilation errors, use repository context, and iteratively refine their outputs, making them more suitable for automatic code generation. [13]–[15]. Tools such as Codex [5] and Cursor [16] are already being integrated into real development environments. However, deployment has outpaced empirical understanding: incidents like Replit's agent deleting a production database[1] illustrate the risks of autonomous systems in critical workflows.

Although agents show strong potential for bug fixing and repository-level tasks, their use for microservice generation remains largely unexplored. Microservice involve numerous fine-grained services communicating together [17], and development tools for supporting microservices are limited [18], [19]. Microservices form a demanding testbed for AI agents because each service must satisfy its own logic while integrating through strict API contracts and interaction patterns. Even small errors can break cross-service communication, and existing tools still struggle with these complexities [17]. Generating microservices therefore requires both local correctness within each service and global correctness across service contracts, making this architectural style a realistic and challenging benchmark for assessing modern AI agent capabilities.

The **goal** of this study is to evaluate the functional correctness and code quality of AI agents-generated microservice. Our study employs 2 prompt types, 4 projects, and 3 AI agents, each generation was done in 2 scenarios and 3 different microservices per project producing 144 implementations evaluated for functionality, code quality, and efficiency across different generation scenarios[2]. We provide the following **main contributions**:

- The first empirical study on AI agents generating microservices under different contextual scenarios, evaluating functional correctness and code quality;
- A comparative analysis of agent efficiency, covering generation time, cost, and token consumption.

Paper Structure: Section II provides background information about AI Agents and related work. Section III describes our research questions, the different Contextual Scenarios in our work and the design of our study. Section IV presents results, which are discussed in Section V along with threats to validity of our study. Finally, Section VI presents our conclusion and future work.

II. Background and Related Work

This section provides an overview of agentic AI systems for code generation and describes related works

[1]https://x.com/amasad/status/1946986468586721478

[2]Code available at: https://doi.org/10.5281/zenodo.17863951

A. Agentic AI for Code Generation

LLM-based agents extend the capabilities of standard Large Language Models (LLMs) with their autonomous task execution capabilities. While foundational LLMs operate as probabilistic engines that predict the next token based on a static input, agents such as OpenAI's Codex [5] wrap these models in a control loop that enables interaction with an external environment [20]. Acode generation agent typically functions through an iterative perception-decision-execution cycle formalized as a trajectory of thoughts, actions, and observations [15]. Within this cycle, the agent utilizes tools, such as file system interfaces and command-line executors to manipulate the environment and generate code. Specifically, the agent receives feedback from these actions (such as compilation errors or test logs etc.) allowing it to validate its assumptions and autonomously refine the implementation [12], [21]. This iterative capability is useful for generating code for microservices, which require precise adherence to file structures and API contracts that may not always be inferred from a single-pass generation.

B. Related Works

Over the years, promising work has been done in specifying software architecture using Architecture Description Languages (ADL) and supporting code generation through model transformations [2], [22]. For microservices specifically, Rademacher et al. [23] proposed an extensible approach to generate microservice code and deployment specifications from LEMMA models. Suljkanović et al. [24] proposed Silveria, a DSL that allows users to model microservice architectures and generate executable code using model transformations. However, as pointed out by Malavolta et al. [2], ADLs in general have a steep learning curve which often hinders practical adoption. Prior to the widespread adoption of LLMs, research on microservice development focused primarily on decomposition which algorithmically identifies optimal service boundaries within monolithic applications [25]. These approaches employ heuristics based on static and dynamic analysis to cluster related components [26]. These works however, are limited by the recommendation for service boundaries and not functional implementation.

Significant work has been done on using LLMs for code generation [27]–[31]. Recent work has shifted toward evaluating autonomous software engineering agents capable of complex, multi-step tasks [32]. The SWE-Bench framework [14] has established test-pass rates as the primary evaluation metric for such agents, using "Fail-to-Pass" tests as the evaluation signal. Research has also evolved from simple Retrieval-Augmented Generation to iterative refinement frameworks [33], where agents generate code, receive compiler feedback, and refine their solutions. The formal analysis of agent trajectories, represented as sequences of (thought, action, result) triples [15], enables researchers to analyze not just final outputs but the reasoning process.

The quality of LLM-generated code remains debated. Some studies find LLM code more complex than human code [34], while others observe low complexity [35]. While other studies find [36] that low complexity reflects missing defensive programming constructs like error handling and input validation. Additionally, benchmark contamination is a recognized methodological problem [37], as popular repositories are likely present in training corpora, potentially inflating performance on well-known benchmarks.

Recent work has demonstrated LLM-based generation of serverless functions as architectural components [10]. We extend this to microservices, which are larger, stateful services with complex integration requirements. To our knowledge, no prior work has systematically evaluated autonomous agents for end-to-end microservice generation across varying contextual scenarios, measuring both functional correctness through automated testing and code quality through established metrics.

III. Study Design

A. Goal

This study aims to evaluate the generation capabilities of AI agents in microservice development across varying levels of contextual information. Using the Goal-Question-Metric approach [38], we formalize our goal to:

analyze the effectiveness of AI agents **for the purpose of** generating microservices **with respect to** functional correctness, code quality and efficiency **from the viewpoint of** software developers and architects **in the context of** existing microservice-based systems.

B. Research Questions

$\mathbf{RQ}_1$: To what extent can AI agents generate microservices across different contextual scenarios, in terms of functional correctness and code quality?

- $\mathbf{RQ}_{1.1}$: To what extent can AI agents develop microservices in **incremental** generation scenarios?
- $\mathbf{RQ}_{1.2}$: To what extent can AI agents develop microservices in **clean state** generation scenarios?

In the incremental generation scenario, We investigate whether AI agents can generate microservices based on requirements specifications, and comprehensive contextual information, including the existing system implementation, documentation, and preserved API integration points (**RQ**1.1). This mirrors the real-world scenario where a team adds a microservice to an already existing system with already running services, established patterns, documented API contracts, and provided templates for configuration.

In clean state generation, we investigate whether AI agents can generate microservices based only on the requirements specifications, without any information of the system implementation (**RQ**1.2). This is similar to the real-world case, which involves teams working from requirements alone on **greenfield projects** or **early-stage development**. Available information **only includes** requirements specifications. Inferring architectural patterns, component interactions, and API designs from minimal context is challenging. This involves significantly more exploration and decision-making compared to incremental

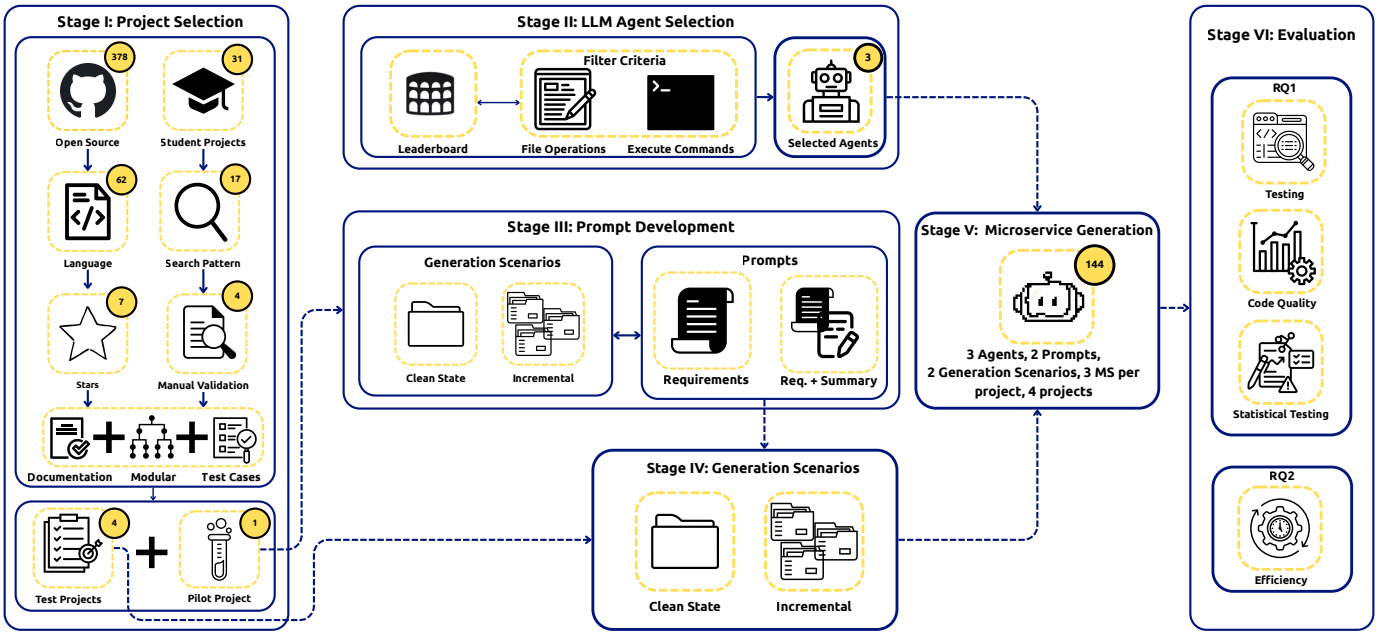

Fig. 1: Overview of the Study Design

generation, where agents can study existing patterns and adapt to them.

We measure **functional correctness** through automated test execution and assess **code quality** through standard software metrics including size (SLOC), complexity (Cyclomatic Complexity), and understandability (Cognitive Complexity).

RQ_2: How do AI agents compare in efficiency when generating microservices?

In this RQ, we compare the resource efficiency of different AI agents by measuring the computational cost (token consumption and associated monetary costs) and time required for generation. This analysis helps determine the practical feasibility and relative performance of using different AI agents for microservice development in real-world scenarios.

C. Experiment Workflow

1) Project Selection: We systematically collected microservice systems from two primary sources to ensure diversity. Our data collection process follows a multi-stage filtering approach to identify suitable repositories for LLM agent evaluation.

Open Source Repository Selection: We began with an existing microservice dataset [39] consisting of 378 sample points and applied a filter based on the patterns seen in Java-based Spring Boot applications, this choice of language was based on familiarity and proper build mechanism. We chose repositories which have over 40% Java content to include repositories that may contain web frontend code (HTML, CSS, JavaScript) as part of a mono-repository structure while having sufficient Java-based microservice backend code for our evaluation, using GitHub API. This gave us 62 repositories. We then filtered repositories based on popularity metrics, requiring systems to have more than 600 GitHub stars to ensure real-world relevance and community validation. This filtering process yielded 7 repositories, from which we further refined our selection to 2 repositories that met our technical requirements and were also widely used in microservice literature [40], [41]. We required systems to provide test cases for functional correctness validation, documentation, and detailed requirements specifications.

Private Project Integration: To complement open source systems which might have been subject to LLM contamination [37], we incorporated private projects from software engineering coursework. In this coursework, students were asked to implement a software system with an architecture of their choice and to provide proper documentation specifying requirements, architectural viewpoints and diagrams. They were also asked to implement an alternative architecture and provide comparison of performance, state pros-cons. Out of the 31 projects we filtered the term ”microservice” (or its alternatives ”micro-service”/”microservices” etc.) in the document, shortlisting 17 candidate projects. On investigation, 4 projects had used the microservice architecture as their main implementation and not as an alternative, these are the ones we selected for further examination. We manually verify that these projects contain documentation and have a modular code structure before finally selecting 3 private projects (2 for evaluation and 1 for pilot study).

Final Dataset Composition: Our complete filtering pipeline resulted in 5 microservice systems with varying levels of complexity. Table I presents the characteristics of the projects.

2) LLM Agent Selection: To evaluate microservice generation capabilities We selected coding agents using established benchmarks while prioritizing both state-of-the-art performance and open-source representation. We select agents from the top

TABLE I: Selected Microservice Projects (O=Open Source, P=Private)

Project	Lang.	Services	Description
PiggyMetrics (O)	Java	3	Personal finance management with account statistics and notifications
Train-Ticket (O)	Java	41	Train ticket booking with search, orders, payment, and user management
TeamSync (P)	Java	4	Team collaboration platform for course registration and task management
Project Management (P)	Python	4	Collaborative task management with hierarchical task breakdown
MicroBank (P, Pilot)	Python	4	Banking system with account management and transaction processing

performers of the LMArena Coding Leaderboard[3] [42]. Our selection prioritized LLMs with available coding agents to perform iterative file operations, execute bash commands, and interact with codebases through tool-calling as microservice generation requires system-level interactions beyond simple code completion. We also use the default coding agents provided by vendors of the LLMs selected, with their default settings and prompts.

The agents selected for microservice generation are described in Table II below. "Access Method" indicates whether the agent requires a paid subscription (Subscription), API token access (API), or can be hosted locally (Local). For agents offering multiple access methods, we highlight the method used in our study in bold. "License Type" refers to whether the model weights are publicly available (Open) or not (Proprietary). All experiments were conducted on a local machine using these coding agents in non-interactive mode with permission bypass flags for uninterrupted execution. We perform the efficiency analysis through the logs of different experimental runs.

TABLE II: Characteristics of Selected LLM Coding Agents

Agent Name	Model	Context Window	Access Method	License Type
Claude Code	claude-sonnet-4 -20250514	1M	**Subscription/** API	Proprietary
Codex	GPT-5 (high effort)	400K	**Subscription/** API	Proprietary
Code Qwen	qwen3-coder -480b-a35b	1M	**API**/Local	Open

3) Pilot Study and Prompt Development: Before conducting the full evaluation, we refined our prompt design through a pilot study using our pilot project with Claude Code. Through this process, we developed a two-stage approach where we generate implementation summaries of existing services using the prompt shown in III-C3. Further, we modified our prompts to include role personification, framing agents as senior engineers. We observed that the generation across multiple runs had similar outputs with this pilot study.

[3]https://lmarena.ai/

Summary Generation Prompt Template

Role: Expert software architect with 15+ years of experience in microservices architecture and enterprise system design, specializing in analyzing complex codebases and distilling them into clear descriptions.

Task: Analyze {*MICROSERVICE_NAME*} and generate a comprehensive description capturing its complete implementation for regeneration purposes.
Instructions: Read requirements document {*REQUIREMENTS_PATH*}, explore codebase to identify all related files, cross-reference implementation with requirements, focus only on this specific microservice (ignore others except interaction points).
Output Format: Generate high-level description including:

- Functional Scope (core responsibilities)
- Key Features Implemented
- API Responsibilities
- Data Responsibilities (managed data and business rules)
- Integrations (connections with other services from this service's perspective)
- Technical Approach (high-level decisions and patterns)
- Implementation Notes (business logic and special considerations)

Prompt Strategies: For each experiment type, we employ two prompt strategies:

- *P1 (Name + Requirements Path)*: Provides only the microservice name and path to requirements document, representing minimal contextual information. Agents must explore the codebase in Incremental or work from requirements alone in Clean State to understand what needs to be implemented.

Minimal Context Prompt (P1)

Role: Senior software engineer with 12+ years experience in distributed systems and microservices architecture.

Task: Generate complete microservice implementation.
Target: {*MICROSERVICE_NAME*}
Requirements: {*REQUIREMENTS_PATH*}
Instructions: Explore codebase, understand patterns and tech stack, generate complete implementation with all components, verify compilation.

- *P2 (Name + Requirements Path + Implementation Summary)*: Includes microservice name, path to requirements document, and a generated implementation summary of the target microservice from III-C3, capturing the microservice's functional scope, API responsibilities, data management patterns, integration points with other services, technical approach, and implementation notes.

Since the requirements contains details about the entire system, the summary highlights the information of the target microservice to be generated to assist the agent.

Prompt with Implementation Summary (P2)

Role: Senior software engineer with 12+ years experience in distributed systems and microservices architecture.

Task: Generate complete microservice implementation.
Target: {*MICROSERVICE_NAME*}
Requirements: {*REQUIREMENTS_PATH*}
Summary: {*GENERATED_SUMMARY from III-C3: Functional scope, API/data responsibilities, integrations, technical approach, implementation notes*}
Instructions: Explore codebase, follow summary guidance, generate from scratch, verify compilation.

4) Generation Scenarios: **Contextual Scenarios:**

We evaluate agents across two generation scenarios representing different levels of system context availability:

- *Incremental Generation ($RQ_{1.1}$)*: The target microservice's code is removed from the codebase, but all integration points and dependencies remain intact, allowing agents to explore the existing architecture before generation. This is done by removing the directory of the microservice since we used mono-repository modular projects. Agents have access to the test cases.
- *Clean State Generation ($RQ_{1.2}$)*: The target microservice implementation traces are removed from the system, including the directory, configuration files, and API calls made to the microservice from other microservices. This is done by locating API calls to external services using search patterns or by referring to documentation for architectural details. Agents must infer system architecture and implementation details from requirements specifications alone without access to existing service implementations.

The original code acts as a baseline to compare metrics for code quality assessment.

5) Microservice Generation: Each experiment type and prompt strategy combination represents a distinct experimental condition, resulting in four conditions total: Inc-P1, Inc-P2, CS-P1, and CS-P2. For each project, we selected three microservices based on their business logic importance and removed them one at a time for generation. For each condition, we evaluate all three selected agents across our four microservice projects, with each project contributing three generation tasks, generating a total of 144 microservice implementations (4 conditions × 3 agents × 4 projects × 3 microservices per project).

6) Evaluation Metrics: For code quality analysis, we used SonarQube throughout the projects, test execution was automated using project-specific testing frameworks: Spring Boot Test for Java projects and unit test for Python projects.

Expected Structure (from baseline unit tests):

```java
package com.trainticket.service;

import com.trainticket.entity.Money;
import com.trainticket.entity.Payment;
import com.trainticket.repository.AddMoneyRepository;
import com.trainticket.repository.PaymentRepository;

public class PaymentServiceImplTest {
    @Mock
    private PaymentRepository paymentRepository;
    @Mock
    private AddMoneyRepository addMoneyRepository;
}
```

Codex Generated Implementation:

```java
// File: payment/entity/MoneyTransaction.java
public class MoneyTransaction { ... }

// File: payment/repository/MoneyTransactionRepository.java
public interface MoneyTransactionRepository { ... }
// Note: Money and AddMoneyRepository classes not generated
```

Fig. 2: Example of structural mismatch in Clean State generation (Train-Ticket payment service): baseline tests expect `com.trainticket.*` package with `Money` and `AddMoneyRepository` classes, but Codex generated `payment.*` package with `MoneyTransaction` and `MoneyTransactionRepository`, causing compilation failures despite functional correctness

Functional Correctness ($RQ_{1.1}$ and $RQ_{1.2}$): We evaluated functional correctness through automated testing, with different strategies for each experiment type:

- *Incremental Generation ($RQ_{1.1}$)*: We executed the original unit test suites associated with each target microservice. Agents had access to these test cases during generation. We calculated test pass rates by dividing passing tests by total tests for the individual microservices.
- *Clean State Generation ($RQ_{1.2}$)*: We assessed integration success by executing tests from other microservices that consume the generated service's APIs. This approach was necessary because agents generating from minimal context often produced functionally correct code with different package structures, class names, and architectural patterns than expected by baseline unit tests. Figure 2 illustrates a representative example from the Train-Ticket payment service, where Codex generated correct payment functionality but the syntactical mismatch causes compilation failures in unit tests despite functional correctness. We measured success by determining whether dependent microservices tests passed without failures.

Code Quality Metrics ($RQ_{1.1}$ and $RQ_{1.2}$): We quantify code quality using metrics that measure size and complexity, comparing generated code against human-written baselines:

- *Source Lines of Code (SLOC)*: Quantifies program size by counting lines containing source code.
- *Cyclomatic Complexity (CC)* [43]: Measures control flow complexity and testability.
- *Cognitive Complexity (CogC)* [44]: Measures code understandability.

For each metric, we calculated the change from baseline across experimental configurations, agents, and projects. We use SonarQube to uniformly assess these metrics, as alternative approaches like CodeBLEU [45] and CodeBERTScore [46] are designed primarily for function-level snippets rather than full projects.

Efficiency Metrics (RQ_2): We measured resource consumption and efficiency:

- *Token Consumption*: Total input and output tokens.
- *Cost*: Associated monetary costs based on each agent's pricing model, using API pricing.
- *Generation Time*: Total time from generation start to completion in minutes.

We use the logs provided by the generation runs to get cost based on the tokens and perform all the experiments under similar network setting.

Statistical Analysis: We first assessed whether the data were normally distributed using the Anderson-Darling (AD) test [47], which evaluates whether a set of observations follows a specified probability distribution by measuring the differences between the empirical cumulative distribution function of the data and the theoretical distribution. Results of the statistical test ($A^2 = 20.068$, $p < 0.001$) allowed us to reject the null hypothesis of normality, confirming that the gathered metrics do not follow a normal distribution.

After establishing non-normality, we employed the Wilcoxon signed-rank [48] test to compare test pass rates across prompt strategies (P1 vs P2). The Wilcoxon signed-rank test is a non-parametric statistical hypothesis test that compares two related samples or repeated measurements to assess whether their population mean ranks differ. We further assessed the magnitude and direction of statistically significant differences using the Dunn-All Pair test [49], which performs pairwise comparisons while controlling for multiple testing. We set the significance level (α) to 0.01, lowering it from the conventional 0.05 to account for the large number of statistical tests performed.

IV. Results

A. $RQ_{1.1}$: To what extent can AI agents develop microservices in incremental generation scenarios?

Functional Correctness: We analyze the test pass rates against the Baseline (B) pass rates of the original project, for incremental generation, where agents had access to the existing system architecture and integration points (Table III). Codex achieved the highest average performance with P1 prompts (75.9%), followed by Claude Code (73.7%) and Code Qwen (50.5%). Providing additional context through P2 prompts and generating the target microservice did not consistently improve performance, with average test pass rates decreasing to 50.3% for Codex, 63.2% for Claude Code, and 47.2% for Code Qwen. In this setting, Code Qwen had comparable results to the other agents. Further, Table III reveals that commercial agents, namely Codex and Claude Code, outperformed the open-source alternative Code Qwen, regardless of prompt type. Notably,

TABLE III: Test Pass Rates for Incremental and Clean State Development for Different Types of Prompts P1 and P2

Agent	Project	Test Pass Rate (%)					
		Incremental			Clean State		
		B	P1	P2	B	P1	P2
Claude Code	Piggymetrics	100.0	91.1	81.7	100.0	98.0	98.0
	Train-Ticket	100.0	92.6	60.6	81.5	97.0	97.0
	TeamSync	100.0	66.7	66.7	100.0	100.0	100.0
	Project-Management	92.3	44.3	43.9	92.3	92.6	96.3
	Avg. Agent Performance	**-**	**73.7**	**63.2**	**-**	**96.9**	**97.8**
Codex	Piggymetrics	100.0	93.8	93.3	100.0	99.0	65.7
	Train-Ticket	100.0	100.0	59.8	81.5	97.0	97.0
	TeamSync	100.0	66.7	0.0	100.0	100.0	100.0
	Project-Management	92.3	43.0	48.0	92.3	96.3	63.0
	Avg. Agent Performance	**-**	**75.9**	**50.3**	**-**	**98.1**	**81.4**
Code Qwen	Piggymetrics	100.0	66.7	84.1	100.0	98.0	98.0
	Train-Ticket	100.0	60.6	25.3	81.5	33.3	97.0
	TeamSync	100.0	33.3	33.3	100.0	100.0	100.0
	Project-Management	92.3	41.5	46.0	92.3	96.3	63.0
	Avg. Agent Performance	**-**	**50.5**	**47.2**	**-**	**81.9**	**89.5**

Codex achieved 100% test pass rate for Train-Ticket (the largest system among the candidates) with P1 prompts, while Code Qwen struggled particularly with TeamSync and Train-Ticket, achieving upto 33.3% pass rates in some configurations.

Code Quality. To assess the quality of generated code beyond functional correctness, we analyzed static code metrics including Lines of Code (LoC), Cyclomatic Complexity (CycC), and Cognitive Complexity (CogC) using SonarQube. Figure 3 presents a comparison of original versus generated code across agents and configurations.

The results show that generated code generally exhibits lower complexity metrics compared to baseline implementations. As shown in the top row of Figure 3, Claude Code and Codex consistently produced code with 20-30% lower cyclomatic and cognitive complexity across all configurations, while maintaining comparable LoC. Code Qwen showed higher variance, particularly in the incremental P2 configuration where complexity metrics approached baseline levels.

The bottom row of Figure 3 presents configuration-level comparisons, revealing that code quality improvements were consistent across different prompt and generation strategies, with all agents demonstrating reductions in complexity across most configurations.

B. $RQ_{1.2}$: To what extent can AI agents develop microservices in clean state generation scenarios?

Functional Correctness. The clean state generation scenario displays different performance patterns. Agents achieved substantially higher average test pass rates: Claude Code (97.8% with P2), Codex (98.1% with P1), and Code Qwen (89.5% with P2) (Table III). Claude Code had the most consistent performance, even achieving 100% integration success for TeamSync regardless of prompt strategy, and maintained 97-98% success rates for Piggymetrics and Train-Ticket. Codex achieved the highest overall performance with P1 prompts (98.1% average), but showed inconsistent results with P2 prompts, particularly dropping to 65.7% for Piggymetrics and

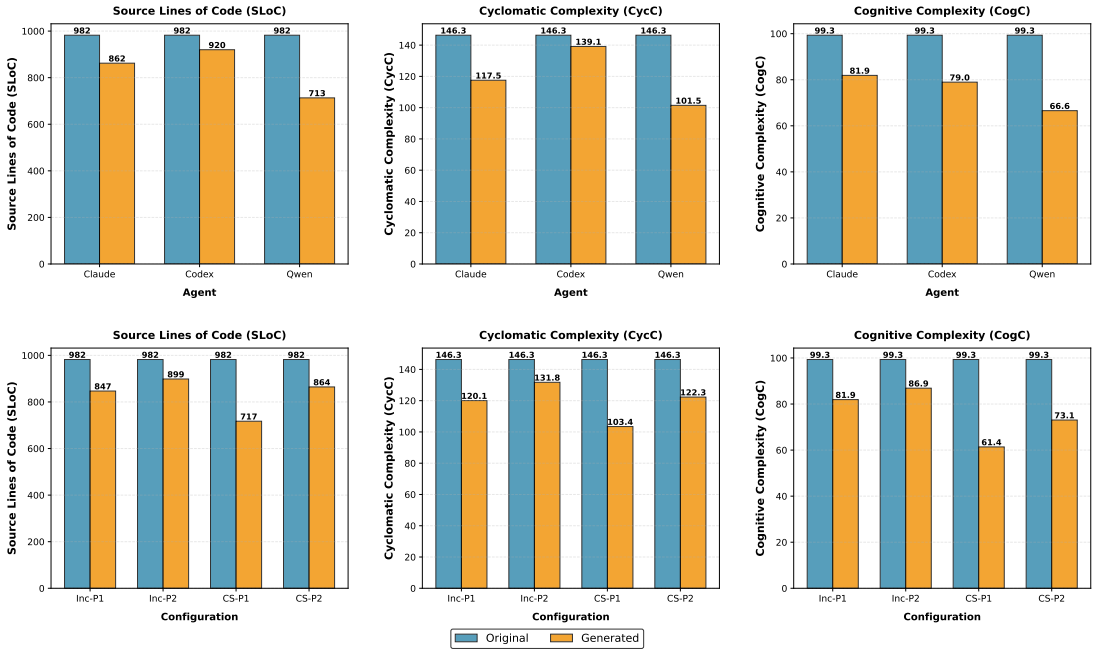

Fig. 3: Code Quality Metrics comparison: Lines of Code (LoC), Cyclomatic Complexity (CycC), and Cognitive Complexity (CogC). Top row shows comparison by agent, bottom row shows comparison by configuration.

63.0% for Project-Management. Different from other agents, Code Qwen failed to write any code in 3 instances with P1, where it just terminated after reading files and realizing its in a loop (for eg- in generating the preserve microservice for Train-Ticket). However, when provided with implementation summaries through P2 prompts, Code Qwen recovered its performance, achieving 97.0% integration success, a 63.7 percentage point improvement. All agents achieved perfect 100% integration success for TeamSync.

Code Quality. The code quality analysis demonstrates that clean state development generally produces simpler code across all agents, with reductions in cyclomatic and cognitive complexity ranging from 15-40% below baseline (Figure 3—denoted by CS-P1, CS-P2). CS-P1 configurations consistently show the largest reductions in code complexity. Claude Code maintained the most consistent quality across both development approaches, while Code Qwen showed variability in the clean state scenarios.

Statistical Significance: To validate observed performance differences, we conducted statistical analysis using non-parametric tests with significance level $\alpha = 0.01$. The Wilcoxon signed-rank test revealed that the performance difference between incremental and clean state generation scenarios is highly significant ($Z = 5.180$, $p < 0.001$). The Dunn-All Pair post-hoc test confirmed this finding, showing clean state achieves a mean difference of 34 percentage points higher test pass rates compared to incremental ($Z = 5.180$, $p < 0.001$). This confirms that the choice of generation scenario ($RQ_{1.1}$ vs $RQ_{1.2}$) has a statistically significant impact on functional correctness. In contrast, prompt type differences (P1 vs P2) did not show statistical significance ($Z = -1.475$, $p = 0.140 > 0.01$), indicating that while numerical variations exist across prompt strategies, these differences could be attributed to random variation or agent-specific characteristics rather than systematic prompt effects.

Main Findings for RQ_1: AI agents can generate functional microservices, but performance varies significantly by context. Incremental generation benefits from minimal prompts that encourage exploration, while clean state generation achieves high pass rates due to integration test flexibility. In clean state scenarios, explicit guidance can help overcome the limitations of the underlying model used. Human oversight remains necessary to ensure API contract compliance.

C. RQ_2: How do AI agents compare in efficiency when generating microservices?

To compare the efficiency of different AI agents for microservice development, we analyze the generation time, computational cost, and token consumption. Table IV presents aggregate statistics across all 48 runs per agent (4 projects × 3 services × 4 configurations).

TABLE IV: Average Efficiency Metrics Summary Across All Configurations (48 runs per agent)

Agent	Avg Time (min)	Total Time (hrs)	Input Tokens (K)	Output Tokens (K)	Avg Cost ($)	Total Cost ($)
Claude Code	7.8	6.20	4,416	2.1	13.28	637.39
Codex	16.6	13.25	4,436	37.5	5.92	284.19
Code Qwen	7.6	6.10	2,916	16.2	2.98	143.08

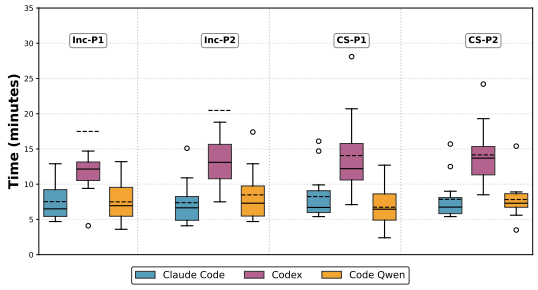

Fig. 4: Time distribution across agents and configurations

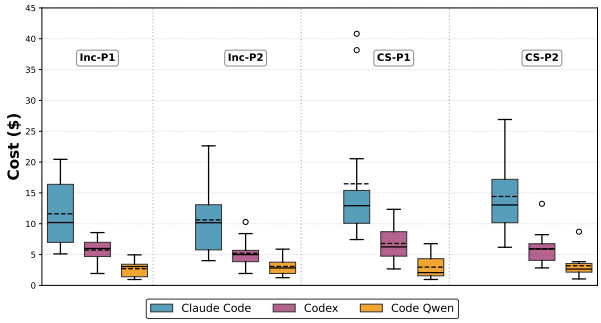

Fig. 5: Cost distribution across agents and configurations

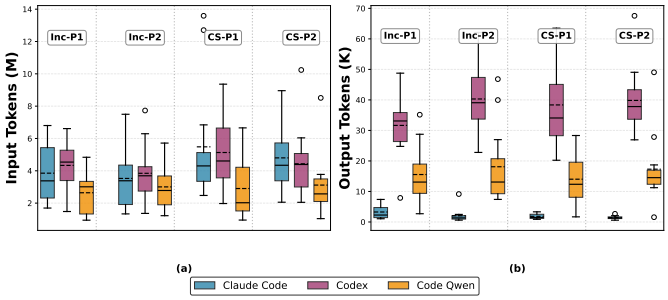

Fig. 6: (a) Input and (b) Output Token distribution across agents and configurations

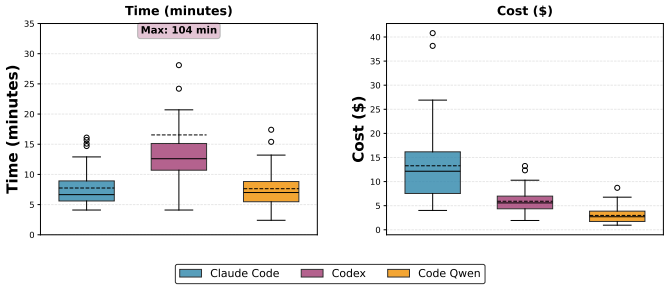

Fig. 7: Time and Cost comparison across agents.

1) Time Efficiency: Figure 4 shows the distribution of generation times across all configurations. Claude Code and Code Qwen demonstrated similar average completion times (7.8 and 7.6 minutes respectively), while Codex required significantly more time with an average of 16.6 minutes per generation task. The maximum observed generation time for Codex reached 1.74 hours for a single microservice, indicating potential timeout concerns for production use.

The time efficiency patterns reveal minimal difference between incremental and clean state generation scenarios for Claude Code and Code Qwen, suggesting these agents efficiently navigate both contexts. Codex showed higher variability, particularly in incremental P2 configurations where additional context increased generation time.

2) Cost Efficiency: Figure 5 presents the monetary cost distribution across configurations. Code Qwen emerged as the most cost-effective option with an average cost of $2.98 per microservice generation, followed by Codex at $5.92 and Claude Code at $13.28. The total cost difference is substantial: generating all 48 microservices cost $143 for Code Qwen, $284 for Codex, and $637 for Claude Code.

Despite higher per-token pricing, Claude Code's significantly lower output token count (averaging 2.1K tokens compared to 37.5K for Codex and 16.2K for Code Qwen) partially offset its higher input token costs. This suggests Claude Code exhibits more efficient token usage in its generation process.

3) Token Consumption Patterns: Figure 6 show input and output token distributions respectively. Input token consumption was comparable between Claude Code (4.4M average) and Codex (4.4M average), while Code Qwen consumed 34% fewer input tokens (2.9M average), likely due to its more efficient context processing.

Output token consumption showed dramatic differences: Claude Code generated remarkably concise outputs (2.1K tokens average), while Codex produced significantly more verbose code (37.5K tokens average). Code Qwen fell between these extremes at 16.2K tokens. When considered alongside the test pass rates from RQ_1, this suggests that verbose output does not necessarily correlate with higher functional correctness - Claude Code achieved comparable or better test pass rates despite generating substantially fewer output tokens.

4) Efficiency Comparison: Figure 7 provides an integrated overview of the primary efficiency dimensions. The time subplot (left) uses a truncated y-axis to highlight how Codex's extreme outliers (reaching 104 minutes) contrast sharply with the 0-20 minute range where most generations occur, while the cost subplot (right) illustrates the substantial cost differences across agents, with Code Qwen being most cost-effective and Claude Code being most expensive.

Main Findings for RQ_2: AI agents show significant differences in efficiency metrics. Claude Code and Code Qwen are faster (7-8 min average) while Codex is slower (16.6 min average) with concerning outliers reaching 1.74 hours. Claude Code is most expensive ($13.28/service) while Code Qwen is most cost-effective ($2.98/service) and generates the most concise code. Token verbosity does not correlate with functional correctness.

V. Discussion

A. Lessons Learned

Explicit implementation summaries degraded incremental generation. Detailed summaries (P2) led to lower unit

P1 Configuration (100% test pass):

```
@Configuration
static class CustomConversionsConfig {
    @Bean
    public CustomConversions customConversions() {
        return new CustomConversions(Arrays.asList(
            new FrequencyReaderConverter(),
            new FrequencyWriterConverter()));
    }
}
```

P2 Configuration (33.3% test pass - missing converters):

```
@Bean
public Clock systemClock() {
    return Clock.systemUTC();
}
// CustomConversions configuration MISSING
```

Fig. 8: Prompt-induced configuration loss in PiggyMetrics notification-service. With minimal prompt (P1), Codex correctly inferred MongoDB converters through codebase exploration (100% test pass). With architectural summary (P2), the agent anchored on high-level patterns (OAuth2, scheduled tasks, service communication) mentioned prominently in the summary while omitting "custom MongoDB converters" mentioned only briefly in implementation notes, causing 12/18 tests to fail with `IllegalStateException`.

test pass rates in incremental scenarios compared to minimal prompts (P1), with Claude Code declining from 73.7% to 63.2% and Codex from 75.9% to 50.3%. Figure 6 reveals P2 prompts consumed fewer input tokens, indicating agents anchored on summaries rather than exploring the repository. Figure 8 illustrates this anchoring effect: for PiggyMetrics notification-service, Codex achieved 100% test pass rate with P1 but degraded to 33.3% with P2 by omitting MongoDB custom converters. While the summary mentioned "custom MongoDB converters for Frequency enum persistence" in implementation notes, it emphasized high-level concerns like OAuth2 integration, scheduled tasks, and service communication patterns. The agent anchored on these prominent features and generated a simplified configuration missing the low-level details it correctly inferred through exploration in P1. However, in clean state scenarios evaluated through integration tests, Code Qwen benefited substantially from explicit guidance. Integration tests only verify API contract compliance rather than internal implementation details, making them more tolerant of structural differences introduced when agents rely on summaries. These agent-specific and scenario-dependent sensitivities indicate that tromping strategies should be empirically determined rather than applying uniform approaches.

Generated microservice implementation is typically more concise than baselines, but efficiency characteristics vary. The microservices generated by agents were generally shorter than human-written baselines (see Figure 3), yet achieved comparable test pass rates. Generation times varied between the agents, with some showing concerning outliers. Agent costs ranged from $2.98 to $13.28 per microservice. The $10 cost difference between the cheapest and most expensive agent is small enough that practitioners should prioritize correctness and reliability over cost efficiency when selecting agents.

Data contamination effects were consistent across all agents. Agents performed substantially better on popular open-source repositories (PiggyMetrics, Train-Ticket) than private projects, likely reflecting training data memorization or contamination. This disparity persisted across proprietary and open-source agents with different architectures, indicating systemic contamination. Our inclusion of student projects suggest that agents may underperform on proprietary codebases relative to benchmark expectations. For researchers, this underscores the necessity of private datasets for unbiased evaluation.

B. Implications for Practice

Agent and prompt selection must be determined empirically rather than following general guidelines. Our results demonstrate no universally optimal configuration across scenarios. In incremental generation, minimal prompts (P1) consistently outperformed detailed summaries for Codex and Claude Code, while Code Qwen required explicit guidance (P2) in clean state scenarios to produce buildable code. Practitioners should conduct structured pilot evaluations: (1) select 2-3 microservices representative of their codebase complexity, (2) test both P1 and P2 prompts with their chosen agent, (3) measure test pass rates and generation time, and (4) establish baseline performance expectations before production deployment [50]. Agent-specific behaviors mean that general prompt engineering advice may not apply, and what works for one agent may degrade performance for another.

Plan for timeout risks and establish generation time limits. Codex showed concerning outliers reaching 104 minutes while the other agents took 7-8 minutes for generation on average. Production systems should implement timeout mechanisms (e.g., 15-20 minute limits) and fallback strategies when generation exceeds acceptable thresholds or agents fail to produce output. For time-sensitive development workflows, select agents with the most predictable performance characteristics. Given that even the most expensive generation costs represent less than 15 minutes of developer time, reliability and correctness should be prioritized over minimizing costs.

Expect performance degradation on proprietary codebases compared to benchmark results. Organizations evaluating agents based on published benchmark results should anticipate lower performance on proprietary codebases, particularly for domain-specific business logic or uncommon architectural patterns. Pilot evaluations on actual production microservices provide more realistic performance expectations than public benchmarks. Additionally, generated code verbosity does not correlate with quality (Fig 3). Concise implementations can achieve comparable correctness to verbose implementations, so practitioners should not interpret brevity as insufficient or verbosity as thorough.

Anticipate evolution toward architecture-specific tooling. Current agents excel at implementation but lack architectural reasoning about organizational context and system-wide trade-offs. Emerging work by Jahića et al. [51] suggests the role of architects is shifting toward evaluating agent-generated

solutions. Organizations should develop evaluation validation processes, as general-purpose agents may lack specialized architectural knowledge for complex system design decisions.

C. Implications for Research

Architecture-specific benchmarks are urgently needed to evaluate agent capabilities. Existing benchmarks like SWE-bench [14] focus on repository-level tasks but lack architectural components with explicit dependencies and API contracts. During our study, we faced challenges finding microservice systems with sufficient documentation and test suites. Future benchmarks should include: (i) diverse architectural styles beyond REST-based microservices, (ii) explicit API contract specifications, (iii) separate unit and integration tests, and (iv) varying complexity levels. Investigating few-shot learning with architectural examples [52] and hybrid context strategies [53], [54] could give insights while designing the prompt.

Methodologies for detecting and mitigating data contamination must be standardized. Agents consistently performed better on popular repositories than private projects across, suggesting memorization rather than reasoning [37]. Since most LLM providers do not disclose training data, future research should develop contamination detection techniques through behavioral analysis.

Human-agent collaboration frameworks require investigation to determine optimal intervention points. Our results show varying success rates depending on context and complexity, yet when should humans intervene remains unclear. Research is needed to identify failure patterns through intermediate artifact analysis, establish risk-based review checkpoints, and develop iterative workflows where humans and agents refine implementations collaboratively. Trajectory analysis [15] provides foundation for understanding agent reasoning, but applying this to identify intervention triggers in architectural generation remains unexplored.

Architecture-specific agents and specialized tooling warrant investigation. Current agents are general-purpose code generators lacking architectural reasoning for system-level concerns like quality attributes and API contracts. Future research should explore specialized agents with architectural tools for contract validation, dependency analysis, and pattern enforcement to better support the complexity of architectural tasks.

D. Threats to Validity

We follow the categorization provided by Wohlin et al. [55] and also provide brief explanation about efforts taken to mitigate the identified threats or why it is not possible, as suggested by Verdecchia et al. [56].

External Validity: Selection of AI agents and generation parameters poses a threat to external validity. However, we systematically selected diverse agents based on LLM provider rankings [42] and architecture-specific capabilities. Generalizability to other architectural patterns beyond REST-based microservices remains limited. Data contamination poses another threat, as the underlying LLMs may have been trained on the open-source repositories used in our study [37]. Since most LLMs are not transparent about their training data, our inclusion of the private projects implemented with lower visibility provides evidence of this effect, as agents consistently performed worse on these repositories compared to popular open-source projects.

Internal Validity: The use of different test strategies for incremental (unit tests) versus clean state (integration tests) scenarios poses a threat, as these validate different aspects of correctness. This distinction was necessary because agents generating from minimal context often produced functionally correct code with different structural implementations that would fail unit tests designed for specific baseline implementations (Fig 2). Further, the use of Claude Code for the pilot study and summary generation poses a threat as it might optimize the performance of it, however Claude Code did not consistently outperform other agents across all scenarios.

Construct Validity: Selection of specific microservices for removal introduces a threat to construct validity. However, we selected 3 microservices per project based on business logic importance. Efficiency metrics were measured under controlled experimental conditions with similar network settings, though variations in cloud service latency may limit generalizability to local development environments. We do not account for prompt caching mechanisms, which may overstate actual costs for agents with effective caching, but all providers we use did not disclose cache-read/write pricing.

VI. Conclusion and Future Work

This study assessed whether AI agents can generate functional and maintainable microservices (RQ_1) and how efficiently they operate (RQ_2). Across 144 generations, agents demonstrated the ability to produce microservices with code quality comparable to or simpler than human baselines. However, performance varied sharply by context: agents can generate functionally correct microservices in $\tilde{6}5\%$ of the cases when expected to be integrated into existing systems ($\mathbf{RQ}_{1.1}$). Conversely, agents could generate functionally correct microservices in $\tilde{9}5\%$ of the cases from requirements alone ($\mathbf{RQ}_{1.2}$), where structural deviations did not hinder integration correctness. These results show that evaluation methodology and context availability strongly shape perceived capability.

Efficiency analysis ($\mathbf{RQ}_2$) revealed large disparities. Claude Code and Code Qwen offered faster and more consistent generation, whereas Codex showed significant variance and costly outliers, and verbosity did not correlate with correctness. Overall, AI agents can support microservice development, yet their reliability remains scenario-dependent, and human oversight is required to ensure API and architectural consistency. Future work should establish architecture-specific benchmarks and clearer human–agent collaboration models to improve robustness and practical adoption.

VII. Data Availability

The complete experimental dataset, including all 144 generated microservice implementations and prompts is available at https://doi.org/10.5281/zenodo.17863951.

REFERENCES

[1] D. Garlan and M. Shaw, "An introduction to software architecture," in *Advances in software engineering and knowledge engineering*, pp. 1–39, World Scientific, 1993.

[2] I. Malavolta, P. Lago, H. Muccini, P. Pelliccione, and A. Tang, "What industry needs from architectural languages: A survey," *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 869–891, 2012.

[3] J. Hutchinson, J. Whittle, M. Rouncefield, and S. Kristoffersen, "Empirical assessment of mde in industry," in *Proceedings of the 33rd International Conference on Software Engineering*, ICSE '11, (New York, NY, USA), p. 471–480, Association for Computing Machinery, 2011.

[4] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," 2024.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.

[6] A. Svyatkovskiy, S. K. Deng, S. Fu, and N. Sundaresan, "Intellicode compose: code generation using transformer," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, (New York, NY, USA), p. 1433–1443, Association for Computing Machinery, 2020.

[7] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," in *The Eleventh International Conference on Learning Representations*, 2023.

[8] M. Esposito, F. Palagiano, V. Lenarduzzi, and D. Taibi, "On large language models in mission-critical it governance: Are we ready yet?," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 504–515, 2025.

[9] M. Esposito, X. Li, S. Moreschini, N. Ahmad, T. Cerny, K. Vaidhyanathan, V. Lenarduzzi, and D. Taibi, "Generative ai for software architecture. applications, challenges, and future directions," 2025.

[10] S. Arun, M. Tedla, and K. Vaidhyanathan, "Llms for generation of architectural components: An exploratory empirical study in the serverless world," 2025.

[11] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, Mar. 2024.

[12] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[13] C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '24, p. 819–831, ACM, Sept. 2024.

[14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?," in *The Twelfth International Conference on Learning Representations*, 2024.

[15] I. Bouzenia and M. Pradel, "Understanding software engineering agents: A study of thought-action-result trajectories," 2025.

[16] Anysphere, Inc., "Cursor ai." https://www.cursor.com, 2023. AI-powered code editor.

[17] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: yesterday, today, and tomorrow," 2017.

[18] H. Vural and M. Koyuncu, "Does domain-driven design lead to finding the optimal modularity of a microservice?," *IEEE Access*, vol. 9, pp. 32721–32733, 02 2021.

[19] C. Zhong, S. Li, H. Huang, X. Liu, Z. Chen, Y. Zhang, and H. Zhang, "Domain-driven design for microservices: An evidence-based investigation," *IEEE Trans. Softw. Eng.*, vol. 50, p. 1425–1449, June 2024.

[20] Y. Dong, X. Jiang, J. Qian, T. Wang, K. Zhang, Z. Jin, and G. Li, "A survey on code generation with llm-based agents," 2025.

[21] J. J. Wu and F. H. Fard, "Humanevalcomm: Benchmarking the communication competence of code generation for llms and llm agents," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, Aug. 2025.

[22] M. Brun, J. Delatour, and Y. Trinquet, "Code generation from aadl to a real-time operating system: An experimentation feedback on the use of model transformation," in *13th IEEE International Conference on Engineering of Complex Computer Systems (iceccs 2008)*, pp. 257–262, IEEE, 2008.

[23] F. Rademacher, J. Sorgalla, P. Wizenty, and S. Trebbau, "Towards an extensible approach for generative microservice development and deployment using lemma," in *European Conference on Software Architecture*, pp. 257–280, Springer, 2021.

[24] A. Suljkanovic, B. Milosavljevic, V. Indjic, and I. Dejanovic, "Developing microservice-based applications using the silvera domain-specific language," *Applied Sciences*, vol. 12, no. 13, p. 6679, 2022.

[25] T. I. Mohottige, A. Polyvyanyy, R. Buyya, C. Fidge, and A. Barros, "Microservices-based software systems reengineering: State-of-the-art and future directions," 2024.

[26] Y. Wang, S. Bornais, and J. Rubin, "Microservice decomposition techniques: An independent tool comparison," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE '24, (New York, NY, USA), p. 1295–1307, Association for Computing Machinery, 2024.

[27] P. Bareiß, B. Souza, M. d'Amorim, and M. Pradel, "Code generation tools (almost) for free? a study of few-shot, pre-trained language models on code," *arXiv preprint arXiv:2206.01335*, 2022.

[28] L. Gong, J. Zhang, M. Wei, H. Zhang, and Z. Huang, "What is the intended usage context of this model? an exploratory study of pre-trained models on various model repositories," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 3, pp. 1–57, 2023.

[29] A. Chen, J. Scheurer, T. Korbak, J. A. Campos, J. S. Chan, S. R. Bowman, K. Cho, and E. Perez, "Improving code generation by training with natural language feedback," *arXiv preprint arXiv:2303.16749*, 2023.

[30] J. Wang and Y. Chen, "A review on code generation with llms: Application and evaluation," in *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, pp. 284–289, IEEE, 2023.

[31] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," 2024.

[32] J. Guo, S. Huang, M. Li, D. Huang, X. Chen, R. Zhang, Z. Guo, H. Yu, S.-M. Yiu, P. Lio, and K.-Y. Lam, "A comprehensive survey on benchmarks and solutions in software engineering of llm-empowered agentic system," 2025.

[33] Z. Bi, Y. Wan, Z. Wang, H. Zhang, B. Guan, F. Lu, Z. Zhang, Y. Sui, H. Jin, and X. Shi, "Iterative refinement of project-level code context for precise code generation with compiler feedback," 2024.

[34] D. G. Paul, H. Zhu, and I. Bayley, "Investigating the smells of llm generated code," 2025.

[35] X. Yu, L. Liu, X. Hu, J. Liu, and X. Xia, "Where is self-admitted code generated by large language models on github?," 2025.

[36] C. J. Chong, Z. Yao, and I. Neamtiu, "Artificial-intelligence generated code considered harmful: A road map for secure and high-quality code generation," 2024.

[37] J. Cao, W. Zhang, and S.-C. Cheung, "Concerned with data contamination? assessing countermeasures in code language model," 2024.

[38] V. R. B. G. Caldiera and H. D. Rombach, "The goal question metric approach," *Encyclopedia of software engineering*, pp. 528–532, 1994.

[39] D. Amoroso d'Aragona, A. Bakhtin, X. Li, R. Su, L. Adams, E. Aponte, F. Boyle, P. Boyle, R. Koerner, J. Lee, F. Tian, Y. Wang, J. Nyyssölä, E. Quevedo, S. M. Rahaman, A. S. Abdelfattah, M. Mäntylä, T. Cerny, and D. Taibi, "A dataset of microservices-based open-source projects," in *Proceedings of the 21st International Conference on Mining Software Repositories*, MSR '24, (New York, NY, USA), p. 504–509, Association for Computing Machinery, 2024.

[40] S. Fischer, P. Urbanke, R. Ramler, M. Steidl, and M. Felderer, "An overview of microservice-based systems used for evaluation in testing and monitoring: A systematic mapping study," in *Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024)*, AST '24, (New York, NY, USA), p. 182–192, Association for Computing Machinery, 2024.

[41] L. Gazzola, M. Goldstein, L. Mariani, I. Segall, and L. Ussi, "Automatic ex-vivo regression testing of microservices," in *Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test*, AST '20, (New York, NY, USA), p. 11–20, Association for Computing Machinery, 2020.

[42] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, B. Zhu, H. Zhang, M. Jordan, J. E. Gonzalez, *et al.*, "Chatbot arena: An open platform for evaluating llms by human preference," in *Forty-first International Conference on Machine Learning*, 2024.

[43] T. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[44] G. A. Campbell, "Cognitive complexity: an overview and evaluation," in *Proceedings of the 2018 International Conference on Technical Debt*, TechDebt '18, (New York, NY, USA), p. 57–58, Association for Computing Machinery, 2018.

[45] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "Codebleu: a method for automatic evaluation of code synthesis," 2020.

[46] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, "Codebertscore: Evaluating code generation with pretrained models of code," 2023.

[47] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd International Conference on Software Engineering*, ICSE '11, (New York, NY, USA), p. 1–10, Association for Computing Machinery, 2011.

[48] L. Yi, G. Gay, and P. Leitner, "An experimental study of real-life llm-proposed performance improvements," 2025.

[49] J. Liu, Q. Huang, X. Xia, E. Shihab, D. Lo, and S. Li, "An exploratory study on the introduction and removal of different types of technical debt," *arXiv preprint arXiv:2101.03730*, 2021.

[50] J. Shin, C. Tang, T. Mohati, M. Nayebi, S. Wang, and H. Hemmati, "Prompt engineering or fine-tuning: An empirical assessment of llms for code," 2025.

[51] J. Jahić, K. Vaidhyanathan, A. Bucaioni, D. Taibi, R. Reussner, S. Krivić, C. Cabrera-Jojoa, T. Sharma, and H. Muccini, "Mind the skills gap: Reimagining the software architect's role in the llm era." Available at SSRN: https://ssrn.com/abstract=5658241, 2025.

[52] Y. Zhang, R. Li, P. Liang, W. Sun, and Y. Liu, "Knowledge-based multi-agent framework for automated software architecture design," 2025.

[53] Y. Shi, Y. Qian, H. Zhang, B. Shen, and X. Gu, "Longcodezip: Compress long context for code language models," 2025.

[54] Z. Yang, S. Chen, C. Gao, Z. Li, X. Hu, K. Liu, and X. Xia, "An empirical study of retrieval-augmented code generation: Challenges and opportunities," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, Aug. 2025.

[55] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, *et al.*, *Experimentation in software engineering*, vol. 236. Springer, 2012.

[56] R. Verdecchia, E. Engström, P. Lago, P. Runeson, and Q. Song, "Threats to validity in software engineering research: A critical reflection," *Information and Software Technology*, vol. 164, p. 107329, 2023.